Regresja logistyczna – funkcja straty i programowanie – Notatki z wykładu (tłumaczenie PL)

Materiał źródłowy: Dr. Brian Mc Ginley (Week 3 - Logistic Regression - Loss Function and Programming.pdf) · Tłumaczenie: Marcin
13 października 2026

Slajdy są w kolejności wykładu. Elementy graficzne, których nie da się przepisać (wykresy, zdjęcia), są opisane w nawiasach kwadratowych. Wzory matematyczne i fragmenty kodu pozostawiono w oryginalnej notacji.

1. Regresja logistyczna: funkcja straty i programowanie (*Logistic Regression: Loss Function & Programming*)

Dr. Brian Mc Ginley

2. Funkcja straty (*Loss Function*)

- Aby wytrenować model regresji logistycznej, musimy ustalić, jakie wagi (w) minimalizują koszt/stratę. Musimy wybrać funkcję kosztu (*cost function*).
- Błąd średniokwadratowy (*Mean Squared Error*) działa bardzo słabo dla regresji logistycznej:

$$(1 / 2m) \cdot \Sigma_{i=1\ldots m} \left(y^{(i)} - f_w(x^{(i)}) \right)^2$$

- Skończyłby się jako funkcja niewypukła (*non-convex*) z wieloma minimami lokalnymi (co utrudnia znalezienie minimum globalnego).

> *[Wykres „Non Convex Graph for Cost Function": oś pozioma – θ, oś pionowa – J(θ); falista zielona krzywa z kilkoma lokalnymi dołkami i jednym głównym minimum, z prawej strony stromo rośnie w górę; pomarańczowa prosta styczna do krzywej w jednym punkcie (zielona kropka); logo GeeksforGeeks w rogu.]*

3. Funkcja straty log loss (*Log Loss Function*)

- Zamiast tego użyj tej funkcji kosztu (modeluje ona stratę dla 1 punktu danych):

$$L(f_w(x), y) = -\log(f_w(x)) \text{ jeśli } y = 1$$
$$L(f_w(x), y) = -\log(1 - f_w(x)) \text{ jeśli } y = 0$$

- Połączmy powyższe równania w jedno:

$$L(f_w(x), y) = -y \cdot \log(f_w(x)) - (1 - y) \cdot \log(1 - f_w(x))$$

- Zwróć uwagę, że to równanie nadal dotyczy tylko 1 próbki.

> *[Wykres funkcji logarytmicznej: oś pozioma x od ok. −1 do 7, oś pionowa y od −3 do 3; krzywa y = log(x) przechodzi przez punkty (1, 0) i (3, 1) oraz punkt o ujemnej wartości y między 0 a 1 na osi x; pionowa przerywana żółto-zielona linia (asymptota) przy x = 0, do której krzywa dąży w dół ku −∞.]*

4. Funkcja straty log loss (*Log Loss Function*) (2)

- Oczywiście musimy obliczyć i uśrednić błąd po wszystkich próbkach. Dlatego jako funkcji straty dla regresji logistycznej używamy poniższej:

$$L(w) = -(1/m) \cdot \left[\Sigma_{i=1\ldots m} \left(-y_i \cdot \log(f_w(x_i)) - (1 - y_i) \cdot \log(1 - f_w(x_i)) \right) \right]$$
(suma po i = 1 … m)

- Funkcja jest wypukła (*convex*), co znacznie upraszcza minimalizację. Musimy znaleźć optymalne parametry **w**, które minimalizują stratę na zbiorze treningowym. Następnie możemy użyć modelu do przewidywania nowych wyników na niewidzianych danych.

- Algorytm rozwiązujący problem i znajdujący **w** jest taki sam jak w innych przypadkach – zobaczymy go później.

5. Programowanie (*Programming*)

```
from sklearn.linear_model import LogisticRegression
```

- Ma kilka opcji, ale w większości zostawimy wartości domyślne.

```
penalty: {'l1', 'l2', 'elasticnet', None }, default='l2'
```

- To jest hiperparametr (*hyperparameter*) – wartość domyślna jest dobra; wspominam o nim tylko na marginesie, wyjaśnię później.

```
maxiter: int, default=100
```

- Kilka razy wspominałem, że uczenie polega na powtarzaniu tych samych czynności w kółko. Tutaj możemy kontrolować maksymalną liczbę powtórzeń tego procesu. Czasami może być potrzeba więcej.

6. Programowanie (*Programming*) (2)

- Teraz o wynikach (outputach). Podobnie jak `LinearRegression`, ma:

```
model.coef_
model.intercept_
model.score(X_test, y_test)       #Gives the “accuracy” score of the model
y_pred=model.predict(X_test)
```

- Nowy element:

```
model.classes_
```

- Zwraca listę etykiet klas znanych modelowi. Np. jeśli Twoje y to:

```
y = np.array([0,1,2,1,1,1,1,1,0,2,1]) #or a pandas  series equivalent
```

- wtedy `model.classes` będzie równe [0, 1, 2]. Jeśli Twoje y to:

```
y = np.array(["cat","dog","hamster","dog","dog","dog","cat"," hamster"])
```

- wtedy `model.classes` będzie zamiast tego równe [“cat”, “dog”, “hamster”].

7. Programowanie (*Programming*) (3)

- Nowa metoda:

```
model.predict_proba(X_ test)
```

- Zwraca oszacowania prawdopodobieństwa dla wszystkich klas. Być może chcesz wiedzieć, jak pewny jest model swoich predykcji – ale pamiętaj, że modele bywają zbyt pewne siebie (*over-confident*)!
- Metoda `.predict`, o której wspomniałem na poprzednim slajdzie, używa progów (*thresholds*) do podejmowania decyzji.
- Możesz też potrzebować prawdopodobieństw do niektórych metryk wydajności, gdy podejmujesz decyzje dotyczące modeli.